Technische Universität Berlin
Faculty IV: Electrical Engineering and Computer Science
Freie Universität Berlin
Department of Mathematics and Computer Science
Algorithmic Bioinformatics

welche uni angeben, welche reihenfolge..?

Automatic Optimization of Level Parameters for RSHash

(RSHash is a minimizer based fast, compressed hash table for storing DNA k-mers. It was recently introduced by the Reinert Group.)

Bjarne David Jehn

Thesis submitted in fulfillment of the requirements for the degree "Bachelor of Science"(B. Sc.) in the field of Computer Science

First Reviewer:	Prof. Dr. Knut Reinert
Second Reviewer:	Prof. Dr. Stephan Kreutzer
Supervisor:	Jonas Schulte-Mattler

Declaration of Authorship

I hereby declare that the thesis submitted is my own, unaided work, completed without any external help. Only the sources and resources listed were used. All passages taken from the sources and aids used, either unchanged or paraphrased, have been marked as such.

Where generative AI tools were used, I have indicated the product name, manufacturer, the software version used, as well as the respective purpose (e.g. checking and improving language in the texts, systematic research). I am fully responsible for the selection, adoption, and all results of the AI-generated output I use.

I have taken note of the Principles for Ensuring Good Research Practice at TU Berlin dated 15 February 2023. https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

I further declare that I have not submitted the thesis in the same or similar form to any other examination authority.

Berlin, _____ _____

Date Signature

Abstract

in English

Zusammenfassung

auf Deutsch

Contents

1 Introduction **6**
- 1.1 RSHash Explanation . . . 8
- 1.2 The Problem of Choosing "Good" Level Parameters . . . 9

2 Notation **11**

3 Sequence Statistics **12**
- 3.1 Minimizer Distribution . . . 12
- 3.2 Average Frequencies . . . 14
- 3.3 Times for Operations on Data Structures . . . 14

4 MINLP Modeling **16**
- 4.1 Model Input . . . 16
- 4.2 Model for One Level . . . 16
  - 4.2.1 Lookup Time . . . 17
  - 4.2.2 Space Consumption . . . 19
  - 4.2.3 Optimization Goal . . . 21
- 4.3 Model for Several Levels . . . 24
  - 4.3.1 Lookup Time . . . 25
  - 4.3.2 Space Consumption . . . 26
- 4.4 Locate . . . 27

5 Experiments **28**
- 5.1 Datasets . . . 28
- 5.2 Considered Parameters . . . 28
- 5.3 Computing the Model Input . . . 29
- 5.4 Model Accuracy . . . 30
  - 5.4.1 Lookup Time . . . 31
  - 5.4.2 Space Consumption . . . 31
- 5.5 Limitations . . . 31

6 Outlook **32**

7 Subsampling **33**
- 7.1 Motivation . . . 33
- 7.2 Subsampling Method . . . 33
- 7.3 Time Complexity of Finding the Minimizer Distributions . . . 33
- 7.4 Time Complexity using Subsampling . . . 33

7.5 Error Analysis 34
7.6 Choosing l 34
7.7 Choosing n 34

8 stuff **35**

Bibliography **37**

1 Introduction

Genetic Sequences

Genetic strands are chains of the nucleobases adenine (A), cytosine (C), guanine (G), thymine (T) and uracil (U). Adenine, cytosine and guanine are present in both DNA and RNA whilst thymine only appears in DNA and uracil is specific to RNA. A genetic sequence can thus be represented as a string over an alphabet of length four meaning a compact representation of two bits needed to store each base. Despite this, genetic databases are often very large as sequences usually consist of several billion bases e.g. approximately $3.1 \cdot 10^9$ bases for the human genomes sequences.

K-mers

The rapid growth of available genetic data due to better and faster sequencing techniques has led to a need for efficient algorithms for processing genetic sequences. Many approaches are based on so called k-mers. The k-mers of a sequence are all its substrings of length k, thus a sequence S with $|S| \geq k$ contains exactly $|S| - k + 1$ k-mers. The sequence $CGACAT$ for example consists of the 3-mers CGA, GAC, ACA, CAT. Uses for k-mers include k-mer counting, frequency analysis, sequence indexing, genome assembly, sequence comparison, taxonomic classification, phylogeny reconstruction, protein sequence searching and alignment as well the construction of k-mer databases [Moe+24].

Lookup and Locate Queries

One important task is the comparison of sequences against large genomic reference datasets by checking whether each k-mer in the sequence of interest is present in the reference dataset, this is referred to as a lookup of the queried k-mers. On the other hand, a so called locate query answers not only whether queried k-mers are present but also returns all positions of the queried k-mers in the sequences. These positions are referred to as their offsets.

Space/Time Trade-Off

Designing a data structure to answer lookup or locate queries comes with an inherent trade of between the time needed for a lookup and the space consumption of the data structure. This can be demonstrated by considering the two extremes for data structures enabling a lookup or locate of a queried k-mer q in a sequence S with $|S| \gg k$ that doesn't contain any duplicate k-mers[1]:

The most space efficient representation of all k-mers in S is the sequence itself needing approximately two bits per k-mer (bpk). A lookup/locate query of a q would require scanning through the whole sequence, comparing every k-mer with q and returning

[1] For sequences containing duplicate k-mers, the lookup case does not change much. However, in the locate case it would then be required to find all offsets of q. For the compact example this would require to always scan through the whole sequence. For the fast example this would mean that the hash table entry for each k-mer would store a list of all its offsets.

whether q appears in the sequence (and its offset for the locate case). Both queries would terminate upon either finding a match or reaching the sequences end, requiring $O(|S|)$ time.
A very fast lookup/locate on the other hand could be achieved by storing the appearance of every k-mer in S in a hash set (locate: hash table) enabling both queries in $O(1)$ time. For this approach every k-mer would have to be stored in the hash set/table leading to a space consumption of $2k$ bits per k-mer in the lookup case and $2k + log_2(|S|)$ bits per k-mer to enable locate queries which require the offsets to be stored as well.
Since both a lookup time in $O(|S|)$ as well as $2k$ bits per k-mer are not viable for large genomes, there has been much work on data structures enabling queries in much less than $O(|S|)$ whilst also maintaining a certain compression.

Minimizers

A common approach to achieve this is the use of so called minimizers. A minimizer of length m is the smallest m-mer of a k-mer regarding a total order over all strings of length m. Using the lexicographical order for example to determine the minimizer of length 2 in the 6-mer $CGACAT$ results in AC because regarding that order, this is the smallest 2-mer in $CGACAT$.
Minimizers have several properties which render them highly interesting for this domain. Firstly they are considered to be sparse because consecutive k-mers are very likely to share the same minimizer. Zheng, Kingsford, and Marçais [ZKM20] show that in a random string S with $|S| \gg k > m$ each minimizer of length m is expected to cover an average of $(k - m + 2)/2$ k-mers. These consecutive k-mers sharing the same minimizer are also referred to as super-k-mers. Even in a sequence only containing unique k-mers, several super-k-mers can have the same minimizer as $m < k$.
A second interesting property is the skewness of the distribution of minimizers in regards to their *frequencies* which is the number of super-k-mers which have a specific minimizer. In a given sequence only a few minimizers have high frequencies whilst most have small frequencies or do not appear at all. Some intuition for this property can be gained by considering an m-mer consisting only of As versus one of Ts. Using the lexicographical order the first will be the minimizer of every super-k-mer in which it appears whilst the second would only become the minimizer of a super-k-mer containing only Ts.
Both the sparsity and skewness of minimizers have been observed by several previous works [Chi+14; Jai+20; Pib22; SMR26].

SSHash

Pibiri [Pib22] for example describes the SSHash (Sparse and Skew hashing of K-mers) algorithm which exploits these two properties by computing all minimizers appearing in a sequence and then use a minimal perfect hash function to store every super-k-mers offsets in an array *Offsets* in the order induced by their minimizers hashes. They use a second array *Sizes* in which the nth element stores the starting index in *Offsets* for the offsets belonging to the nth minimizer. The sparsity of minimizers mainly effects the data structures space consumption: By indexing super-k-mers, they can compress the data structures size by a factor of $(k - m + 2)/2$ compared to indexing individual k-mers. The skewness of minimizers on the other hand is of importance to the locate time: Querying a k-mer q requires computing its minimizer and then searching for it in

every super-k-mer with the same minimizer. For a sufficiently large minimizer length, minimizers of high frequency become increasingly unlikely.
The both space consumption and locate time of SSHash are highly dependent of the minimizer length m: A smaller m means a higher compression $(k - m + 2)/2$. The lookup/locate time on the other hand is largest for small m because those result in minimizers of higher *frequencies*. This demonstrates the challenge that comes with choosing an optimal minimizer length.

1.1 RSHash Explanation

Notiz: Im Fokuss sollte hier stehen, zu erklären was es bedeutet frequente minimizer abzuschneiden, wie davon der speicherplatz und die Laufzeit abhängen und wie mehrere level dabei helfen können.
- Wird das erfüllt?

RSHash is a minimizer based fast, compressed hash table for storing DNA k-mers that answers lookup and can be extended for supporting locate. It was recently introduced by Schulte-Mattler and Reinert [SMR26]. It follows a minimizer based approach like SSHash; however, instead of using minimal perfect hashing, RSHash relies on bitvectors supporting rank and select queries. Its main difference to previous methods is the introduction of a layered multi-minimizer scheme aiming to reduce the cost of high *frequencies* whilst at also maintaining large compression.
The data structure works with one, two or three so called levels and a hash table. In order to build RSHash to answer lookup (or locate) queries one has to specify the number of levels as well as a pair of level parameters for each level l: the minimizer length m^l and a threshold t^l.

When using only one level, that level is build by first computing each k-mers minimizer of length m^1 using a hash function h_1 and counting the number of times each minimizer occurs in the sequences. We call minimizers that appear at least once but no more than the threshold t^1 to be *infrequent* on the level.
A bitvector R^1 of lengths 4^{m^1} is used to mark the appearance of each distinct *infrequent* minimizers. For this, the minimizers are interpreted as binary numbers of $2m^1$ bits and serve directly as indices to the vector. If for example the minimizers length were chosen to be $m^1 = 2$ and AA was an *infrequent* minimizer in the sequences, then $R^1[AA] = R^1[0]$ would be set to one.

further explain the use of the hash function?

The offsets each *infrequent* minimizers occurrences in the sequences are stored in the offset vector O^1 in the order induced by R^1. Let f for example be the frequency of the smallest *infrequent* minimizer, that is the number of occurrences of the smallest minimizer for which the bit in R^1 is set. Then $O^1[0 : n)$ contains the offsets of that minimizer whilst the next offsets $O^1[n : ...)$ belong to the second smallest minimizer and so forth.
The boundaries of each minimizers offsets in O^1 are marked in the bitvector S^1 of length $|O^1| + 1$. Following up on the example above, the first bit $S^1[0]$ would be set to one, indicating the start of the offsets belonging to the smallest minimizer, this would be

followed by zeros until the nth bit $S^1[n] = 1$ indicating the start of the offsets belonging to the second smallest minimizer and so forth. The last bit in S is also set to one marking the end of the largest minimizers offsets.

If a second level is specified, only those super-k-mers with *frequent* minimizers on the first level are handled on the second level which is build almost identically to the first level, creating the vectors R^2, O^2 and S^2 but using the hash function h_2 for the computation of minimizers. The use of a different hash function for the computation leads to a skewed distribution of minimizers, in which most k-mers having *frequent* minimizers on the first level, have *infrequent* ones on the second. This holds even when the specified minimizer length m^2 and threshold t^2 are equal to those on the first.
A third level would be build analogously, if specified.

Only the offsets of k-mers that have *frequent* minimizers on all specified levels are then directly stored in a fast hash table sacrificing some space in return for a fast lookup which is viable as those k-mers are only a small fraction of the whole input sequence for a good choice of the level parameters.

1.2 The Problem of Choosing "Good" Level Parameters

We already noted that using a small $m \ll k$ thus leads to a relatively small number of super-k-mers meaning a better compression, because each minimizer is expected to be valid for an average of $(k - m + 2)/2$ k-mers [ZKM20]. However, as described, a small m also means that minimizers will be more *frequent* which can lead to high query times when each occurrence of a queried k-mers minimizer has to be evaluated for an actual match with the k-mer itself. This trade off between space consumption and query times both dependent of the chosen parameters is an inherent problem of minimizer based k-mer indexing methods. Existing methods use a number of approaches to deal with this problem like Pibiri [Pib22] who choose a relatively large minimizer length, accepting a smaller compression in order to maintain reasonable locate times.

The leveled approach introduced in the RSHash algorithm on the other hand aims to achieve high compression by using relatively short minimizers on the lower levels whilst at the same time mitigating the burden of very *frequent* minimizers by using thresholds to cut them off and guaranty fast queries on every level. The use of distinct hash function to compute minimizers on each level leads to different skewed distributions of the minimizers on each level. Because only a small percentage of the k-mers are frequent for each distribution, the number of k-mers reduces drastically with each level and only a small percentage of the sequences k-mers end up being stored in the spatially expensive hash table.

The RSHash approach thus allow significantly faster queries opposed to SSHash whilst remaining similarly compact. However, this is highly dependent on the chosen minimizer lengths which influence the sparsity and skewness of minimizer distributions on each level and the chosen thresholds, which affect both the lookup (or locate) time on each level as well as the percentage of k-mers having to be stored in the hash table. For example, choosing very short minimizers and small thresholds on every level would result in a

very large hash table as almost no minimizer would be *infrequent* .

Choosing "good" level parameters to build the data structure for a given set of sequences requires manual testing and extensive understanding of the RSHash algorithm.

We will attempt to expand the understanding of how exactly the level parameters influence the time and space consumption of RSHash and make the algorithm more accessible to use by automating the optimization of level parameters for a given genome and optimization goal.

2 Notation

A sequence is a string over the DNA alphabet $\Sigma = \{A, C, G, T\}$. Let $i, j \in \mathbb{N}$ with $i < j$ be zero based indices of a string S of length $|S| \geq j$. We use $S[i : j)$ to denote the substring from the ith to the $(j-1)$th character of S. The k-mers of a string S with $|S| \geq k$ are all its substrings $S[i : i + k) \in \Sigma^k$ of length k for $i \in \mathbb{N} \wedge i \leq |S| - k$.
Let $L \in \mathbb{N}^+$ be the number of levels specified. We refer to the set of levels $\mathcal{L} := \{l \in \mathbb{N}^+ \mid l \leq L\}$. We use $\mathcal{M} \subseteq [1, k) \cap \mathbb{N}$ and $\mathcal{T} \subset \mathbb{N}^+$ to denote the sets of minimizer lengths and thresholds considered. We denote the Cartesian nth power of a set X with X^n e.g. $X^3 = X \times X \times X$. A chosen minimizer length and threshold on a specified level $l \in \mathcal{L}$ are denoted m^l and t^l.
We use the tuple $p^l := (m^1, ..., m^l, t^1, ..., t^l) \in \mathcal{M}^l \times \mathcal{T}^l$ to denote the level parameters up to level l. For a specified tuple p^l of level parameters we use p^{l-1} to denote the level parameters $(m^1, ..., m^{l-1}, t^1, ..., t^{l-1})$ up to the previous level. Furthermore, we introduce the operator $p^l[m^i] = m^i$ to denote retrieving the minimizer length specified for level i from the level parameters and analogously $p^l[t^i] = t^i$.
We write $S_{p^{l-1}} := \{S_1^l, S_2^l, ..., S_n^l\}$ to denote the set of sequences on level l in dependence of the level parameters p^{l-1} on the previous levels i.e. the k-mers which have *infrequent* minimizers on all previous levels. The full set of input sequences when building RSHash are denoted with S_{p^0} and do not depend on any parameters.
Let $q \in \Sigma^k$ be a k-mer. The computation of its minimizer $min_{m^l}^{h_l}(q)$ on level l depends both on the specified minimizer length m^l as well as the hash function h_l on that level, which is not dependent of the level parameters but unique for every level. We denote the minimizers frequency with $freq(min_{m^l}^{h_l}(q))$ which tells the combined number of the minimizer occurrences in a given set of input sequences.
We call the minimizer of q *infrequent* on level l only if $1 \leq freq(min_{m^l}^{h_l}(q)) \leq t^l$ holds i.e. the minimizer occurs at least once but no more than the specified threshold on level l. Minimizers with frequencies larger than the threshold are referred to as being *frequent* .

3 Sequence Statistics

As outlined in the Chapter 1, the space consumption and query time of RSHash are highly dependent on the choice of level parameters as those influence the distributions of minimizers on all levels. We define aspects of the minimizer distribution which will be needed for modeling the space consumption and query time in Chapter 4. Specifically relevant for this is the behavior of minimizer distributions across several levels, which is unique to the approach proposed by Schulte-Mattler and Reinert [SMR26] and therefor not considered in previous works.

3.1 Minimizer Distribution

Recall that we use $p^l \in \mathcal{M}^l \times \mathcal{T}^l$ to describe the level parameters on all levels up to and including the level l, that $S_{p^{l-1}}$ is the the set of input sequences on level l which depends on the level parameters p^{l-1} on all previous and that S_{p^0} is the full set of input sequences on the first level, which is independent of any parameters.

Let $\mathcal{K}_{p^{l-1}}$ be the set of distinct k-mers in a set of sequences $S_{p^{l-1}}$. In other words, the set $\mathcal{K}_{p^{l-1}}$ contains every distinct substring of length k which appears in at least one of the sequences $S_i^l \in S_{p^{l-1}}$ which depend on the level parameters on previous levels:

$$\mathcal{K}_{p^{l-1}} := \{q \in \Sigma^k \mid \exists S' \in S_{p^{l-1}} \; \exists (1 \leq i \leq |S'| - k) \; S'[i : i + k) = q\} \tag{3.1}$$

Let $\mathcal{M}_{p^{l-1},m^l}$ be the set of distinct minimizers of lengths m^l on level l of the k-mers in $\mathcal{K}_{p^{l-1}}$. The set of distinct minimizers is not only affected by the choice of level parameters on previous levels but also depends on the minimizer length on the current level:

$$\mathcal{M}_{p^{l-1},m^l} := \{min_{m^l}^{h_l}(q) \mid \forall q \in \mathcal{K}_{p^{l-1}}\} \tag{3.2}$$

Let $M[p^{l-1}, m]$ be a discrete random variable that describes the frequency of these minimizers:

$$M[p^{l-1}, m] : \mathcal{M}_{p^{l-1},m^l} \to \mathbb{N} \tag{3.3}$$

$$M[p^{l-1}, m](x) = freq(x) \tag{3.4}$$

We define its probability distribution $p_{M[p^{l-1},m^l]}$[1], which tells the probability of a minimizer having an exact frequency $f \in \mathbb{N}$. The distribution of minimizers depends both on

[1]This notation differs slightly from that used by Schulte-Mattler and Reinert [SMR26] who do not introduce the probability distributions but instead use p_M and p_K to denote the cumulative distribution functions which we denote with F_M and F_K

the level parameters p^{l-1} on previous levels, which affect the remaining k-mers on level l and the minimizer length m^l on the current level, which directly affects the computation of minimizers. We thus index the distributions with those parameters:

$$p_{M[p^{l-1},m^l]} : \mathbb{N} \to [0,1] \tag{3.5}$$

$$p_{M[p^{l-1},m^l]}(f) = \mathbb{P}(M[p^{l-1},m^l] = f) = \frac{|\{x \in \mathcal{M}_{p^{l-1},m^l} \mid M[p^{l-1},m^l](x) = f\}|}{|\mathcal{M}_{p^{l-1},m^l}|} \tag{3.6}$$

Using the random variables probability function we can derive its cumulative distribution function $F_{M[p^{l-1},m]}$, which returns the probability of a minimizer having a frequency that does not exceed an upper bound f:

$$F_{M[p^{l-1},m^l]} : \mathbb{N} \to [0,1] \tag{3.7}$$

$$F_{M[p^{l-1},m^l]}(f) = \mathbb{P}(M[p^{l-1},m^l] \leq f) = \sum_{i=1}^{f} p_{M[p^{l-1},m^l]}(i) \tag{3.8}$$

Whilst $M[p^{l-1},m^l]$ and its distribution are useful for modeling values depending of a random (appearing) minimizer, RSHash is used to lookup k-mers, not minimizers. Therefore, we introduce the discrete random variable $K[p^{l-1},m^l]$ which is defined for every distinct (appearing) k-mer on the level and describes the frequency each k-mers minimizer dependent of the level parameters p^{l-1} on all previous levels and the minimizer length m^l on the current one:

$$K[p^{l-1},m^l] : \mathcal{K}_{p^{l-1}} \to \mathbb{N} \tag{3.9}$$

$$K[p^{l-1},m^l](q) = freq(min^{h_l}_{m^l}(q)) \tag{3.10}$$

Its probability distribution $p_{K[p^{l-1},m^l]}$ and its cumulative distribution function $F_{K[p^{l-1},m^l]}$ are:

$$p_{K[p^{l-1},m^l]} : \mathbb{N} \to [0,1] \tag{3.11}$$

$$p_{K[p^{l-1},m^l]}(f) = \mathbb{P}(K[p^{l-1},m^l] = f) = \frac{|\{q \in \mathcal{K}_{p^{l-1}} \mid K[p^{l-1},m^l](q) = f\}|}{|\mathcal{K}_{p^{l-1}}|} \tag{3.12}$$

$$F_{K[p^{l-1},m^l]} : \mathbb{N} \to [0,1] \tag{3.13}$$

$$F_{K[p^{l-1},m^l]}(f) = \mathbb{P}(K[p^{l-1},m^l] \leq f) = \sum_{i=1}^{f} p_{K[p^{l-1},m^l]}(i) \tag{3.14}$$

Let p^l be a combination of level parameters on a level l. Recall that we use $p^l[m^l]$ to denote retrieving the minimizer length specified for level l from the level parameters and analogously for $p^l[t^l]$. For better readability we introduce a more compact notation to denote the probability $F_{M[p^l]}$ of a minimizer being *infrequent* on level l and $F_{K[p^l]}$ of a k-mer having an *infrequent* minimizer on that level, both in dependence of all level parameters:

$$F_{M[p^l]} = F_{M[p^{l-1},p^l[m^l]]}(p^l[t^l]) \tag{3.15}$$

$$F_{K[p^l]} = F_{K[p^{l-1},p^l[m^l]]}(p^l[t^l]) \tag{3.16}$$

Explain how and why distribution on the first level affects the distributions on higher levels. Aber nur kurz mit verweis -¿ Experiments

3.2 Average Frequencies

Let $\Omega_{M[p^{l-1},m^l]}$ be the sample space of the random variable $M[p^{l-1}, m^l]$ i.e. the set which contains all frequencies f for which there is at least one minimizer x on level l which occurs exactly f times:

$$\Omega_{M[p^{l-1},m^l]} = \{f \mid \exists x \in \mathcal{M}_{p^{l-1},m^l} freq(x) = f\} \tag{3.17}$$

The average frequency of minimizers on a level $l \in \mathcal{L}$ is given by the expected value of the probability distribution of the random variable $M[p^{l-1}, m]$:

$$\mathbb{E}[M[p^{l-1}, m^l]] = \sum_{f \in \Omega_{M[p^{l-1},m^l]}} p_{M[p^{l-1},m^l]}(f) * f \tag{3.18}$$

For a given set of finite sequences the set of distinct minimizers on each level and for each combination of level parameters is finite by definition. It follows that for a given sequence and set of level parameters, the average frequency $\mathbb{E}[M[p^{l-1}, m^l]]$ can always be computed, because the sample space $\Omega_{M[p^{l-1},m^l]}$ is then finite by definition.

More relevant to the optimization of level parameters is the average frequency of *infrequent* minimizers i.e. the average frequency of minimizers whose frequency do not exceed the threshold t^l. We denote this value with $f_{avg}(p^l)$ and can derive it from $p_{M[p^{l-1},m]}$ by computing a weighted sum over all frequencies up until the threshold t^l and then norming the result by dividing through the percentage of *infrequent* minimizers:

$$f_{avg}(p^l) = \frac{1}{F_{M[p^l]}} \cdot \sum_{f=1}^{t^l} p_{M[p^{l-1},p^l[m^l]]}(f) \cdot f \tag{3.19}$$

3.3 Times for Operations on Data Structures

-¿ Experimente??

We denote the time needed to perform an operation *op* on a data structure *struct* with $T_{op_{struct}}$ e.g. the time for a select query on the bitvector S is T_{sel_S}.

incomplete. measure or model?

Vector Access Accessing an element of a Vector can be done in constant time, we use the following values based on measurements:

$$T_{contains_R} = ...$$
$$T_{contains_O} = ...$$

Hash Table Access Accessing an element of a hash table can be done in constant time, we use the following values based on measurements:

$$T_{check_H} = ...$$

Bit Vector Rank Query A bitvector of length U with n $1s$ can be encoded with $n\lceil log_2 U/n \rceil + 3n$ many bits supporting rank in $O(log_2(U/n)$ time [OS07].

does that help here? Doesn't the model require actual time instead of O notation - maybe measurements would be more suited

Bit Vector Select Query ???
Comparison with Text ???

4 MINLP Modeling

We model the optimization problem as a Mixed Integer Non Linear Programming (MINLP). In Section 4.2 we model the runtime and space needed by RSHash on one level in dependence of the level parameters and minimizers distribution. We then extend this to the case of several levels in Section 4.3.
In Chapter 5 we describe how to find the optimal level parameters for a given sequence using the MINLP.

4.1 Model Input

The set of levels $\mathcal{L}$, the sets of minimizer lengths $\mathcal{M} \subseteq [1, k) \cap \mathbb{N}$ and thresholds $\mathcal{T} \subset \mathbb{N}$ to consider have to be specified explicitly.
From the computed minimizer statistics, the values $F_{M[p^{l-1},p^l[m^l]]}(t^l)$, $F_{K[p^{l-1},p^l[m^l]]}(t^l)$ and $f_{avg}(p^l)$ for all possible combinations p^l of the level parameters on each level $l \in \mathcal{L}$ also have to be passed as input to the MINLP.
Lastly, a measure of the times $T_{contains_R}$, T_{sel_S}, $T_{contains_O}$, T_{comp_T}, T_{check_H} needed for the contains query on bitvector R, the select queries on bitvector S, a comparison with the text T at a random offset and checking whether a key exists in the hash set H. The computation of the minimizer distribution as well as the times needed to perform these actions on the specified data structures is outlined in Chapter 5.

4.2 Model for One Level

Aiming to improve readability, we omit the superscript index indicating an instances belonging to a specific level as only one level is considered in this section e.g. the minimizer length is simply referred to as m instead of m^1.
In order to determine $\mathbf{m}$ and $\mathbf{t}$ we use binary indicator variables μ_m and τ_t for each possible $m \in \mathcal{M}$ and $t \in \mathcal{T}$:

$$\mu_m \in \{0, 1\} \mid \forall m \in \mathcal{M} \tag{4.1}$$
$$\tau_t \in \{0, 1\} \mid \forall t \in \mathcal{T} \tag{4.2}$$

The indicator variables for each minimizer length should be mutually exclusive as $\mathbf{m}$ can only take one distinct value. The same holds for the indicator variables for each

threshold:

$$\sum_{m \in \mathcal{M}} \mu_m = 1 \tag{4.3}$$

$$\sum_{t \in \mathcal{T}} \tau_t = 1 \tag{4.4}$$

The variables $\mathbf{m} \in \mathcal{M}$ and $\mathbf{t} \in \mathcal{T}$ are introduced to describe the optimal minimizer length and threshold.
There values can be modeled through a sum over the indicator variables multiplied with there corresponding values:

$$\mathbf{m} \in \mathcal{M} \tag{4.5}$$

$$\mathbf{t} \in \mathcal{T} \tag{4.6}$$

$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m \cdot m \tag{4.7}$$

$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t \cdot t \tag{4.8}$$

We introduce the variable $\mathbf{F}_M \in [0, 1]$ to take the value of $F_{M[(\mathbf{m},\mathbf{t})]}$ i.e. the percentage of minimizers of length $\mathbf{m}$ in the input sequence, whose frequencies do not exceed $\mathbf{t}$ (Section 3.1).
It is not possible to directly evaluate this value as both $\mathbf{m}$ and $\mathbf{t}$ are variables of the model. Instead we use the indicator variables to model a constraint forcing $\mathbf{F}_M$ to take the value of $F_{M[(m,t)]}$ for which $\mu_m = 1$ and $\tau_t = 1$:

$$\mathbf{F}_M \in [0, 1] \tag{4.9}$$

$$\mathbf{F}_M = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} F_{M[(m,t)]} \cdot \mu_m \tau_t \tag{4.10}$$

Analogously we introduce $\mathbf{F}_K \in [0, 1]$ to represent $F_{K[(\mathbf{m},\mathbf{t})]}$ and $\mathbf{f_{avg}} \in \mathbb{R}_0^+$ to represent $f_{avg}((\mathbf{m}, \mathbf{t}))$ (Section 3.2):

$$\mathbf{F}_K \in [0, 1] \tag{4.11}$$

$$\mathbf{f_{avg}} \in \mathbb{R}_0^+ \tag{4.12}$$

$$\mathbf{F}_K = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} F_{K[(m,t)]} \cdot \mu_m \tau_t \tag{4.13}$$

$$\mathbf{f_{avg}} = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} f_{avg}((m,t)) \cdot \mu_m \tau_t \tag{4.14}$$

4.2.1 Lookup Time

note that this is the average lookup time for k-mers that appear in the sequence! k-mers that do not appear in the sequence will simply have a lookup time of a check in R on every level and then a check in H

In order to model the time needed for a lookup using RSHash, we closely follow the lookup procedure as described by Schulte-Mattler and Reinert [SMR26]:
A k-mer q is looked up by first computing its minimizer $min_m^h(q)$ and checking whether the bit $R[min_m^h(q)]$ is set whilst simultaneously computing its rank $r = rank_R(min_m^h(q))$, taking $T_{contains_R}$ time (part of the model input Section 4.1).
For the percentage $\mathbf{F}_K$ (Equation (4.13)) of the k-mers this bit is set and the lookup is completed by searching for q on the level. We introduce the variable $T_{level} \in \mathbb{R}_0^+$ to describe the time needed for this.
If the bit $R[min_m^h(q)]$ is not set, the lookup is completed through a lookup in the hash table H taking T_{check_H} time (part of the model input Section 4.1). The variable $T_{lookup_{RSHash}} \in \mathbb{R}_0^+$ describing the lookup time can thus be modeled as:

$$T_{level} \in \mathbb{R}_0^+ \tag{4.15}$$

$$T_{lookup_{RSHash}} \in \mathbb{R}_0^+ \tag{4.16}$$

$$T_{lookup_{RSHash}} = T_{contains_R} + \mathbf{F}_K \cdot T_{level} + (1 - \mathbf{F}_K) \cdot T_{check_H} \tag{4.17}$$

The search for those k-mers q for which the bit $R[min_m^h(q)]$ is set continues on the level by using the minimizers rank $r = rank_R(min_m^h(q))$ in the select queries $first = sel_S(r)$ and $last = sel_S(r+1) - 1$ which tell the indices in Offsets O where the minimizer's first and last offset in the text T are stored. These two queries take $2T_{sel_S}$ time.
explain select and rank
Finally the presence of q in T is checked at every minimizer offset in the interval $O[first : last]$ until q is either found or the end of the interval is reached. The average frequency $\mathbf{f_{avg}}$ of *infrequent* minimizers is exactly the average length of this interval. In the lookup case, the comparison is terminated as soon as q is found for the first time, which is on average the case after $(\mathbf{f_{avg}} + 1)/2$ comparisons ($+1$ is necessary because the first index always has to be checked) with each comparison taking T_{comp_T} time.
The following constraint models the complete time T_{level} needed to search for a k-mers presence on the level:

$$T_{level} = 2T_{sel_S} + (\mathbf{f_{avg}} + 1)/2 \cdot (T_{contains_O} + T_{comp_T}) \tag{4.18}$$

access and rank of R are computed simultaneously - combine $T_{R_{acc}}$ and $T_{R_{rank}}$ and move them out of $T_{level_{arg2}}$

Without further constraints, solving the described MINLP for minimal $T_{lookup_{RSHash}}$ results in a solution where the μ_m, τ_t corresponding to the smallest $m \in \mathcal{M}$, $t \in \mathcal{T}$ will be set to one.
This is expected, as this leads to the smallest possible $\mathbf{F}_K$ which minimizes the k-mers having to be looked up in the text and maximizes the lookups in the (faster) hash table. However, this results in maximal space consumption, as each entry in the hash table is significantly larger than an entry in O as described in Section 1.2.
In order to account for that, we have to add constraints modeling the space consumption.

4.2.2 Space Consumption

Instead of modeling the total space used by RSHash we will focus on the bits per k-mer (bpk) used by the data structure as this metric allows to compare compression independent of the sequence length.
The space consumption of RSHash can be decomposed into the space needed for the data structures components: the text T, the vectors R, S and O as well as the hash table H. We use one variable to describe the bits per k-mer needed by each of the components:

$$bpk_T \in \mathbb{R}_0^+ \tag{4.19}$$
$$bpk_R \in \mathbb{R}_0^+ \tag{4.20}$$
$$bpk_S \in \mathbb{R}_0^+ \tag{4.21}$$
$$bpk_O \in \mathbb{R}_0^+ \tag{4.22}$$
$$bpk_H \in \mathbb{R}_0^+ \tag{4.23}$$

The text T is much longer than k. Because of this the bits per k-mer for storing the text T can be set to a constant of 2 bits needed to store one base:

$$bpk_T = 2 \tag{4.24}$$

larger for unitigs!

Bitvector R is a bitvector of length $4^{\mathbf{m}}$ in which each bit set to 1 marks the occurrence of one distinct *infrequent* minimizer in the sequences. We can compute the number n_{DIM} of distinct *infrequent* minimizers by multiplying the percentage of *infrequent* minimizers with the total number of minimizers divided through the average frequency[1] of each minimizer:

$$n_{DIM} \in \mathbb{R}_0^+ \tag{4.25}$$
$$n_{DIM} = \frac{\mathbf{F}_K \cdot n_{kmers}}{\mathbf{f_{avg}}} \cdot \frac{2}{k - \mathbf{m} + 2} \mathbf{OR} \tag{4.26}$$
$$= \frac{\mathbf{F}_M \cdot n_{minimizers}}{\mathbf{f_{avg}}} \tag{4.27}$$

decide what to use

add n_{kmers} (only total?) and $n_{minimizers}$ (for every level in depence of level parameters) to the Model

figure out how to model the ratio between k-mers and minimizers for fractured input sequences OR use $n_{minimizers}$?

Using Elias-Fano-Encoding, a bitvector of size U with n bits set to one can be compressed to $n\lceil log_2(U/n)\rceil + 3n$ [OS07] . Compressing R and dividing this by the total

woher kommt diese formel, kann sie so im paper nicht finden

number n_{kmers} of k-mers yields the constraint describing the bits per k-mer of R:

$$bpk_R = \frac{1}{n_{kmers}} \cdot \left(n_{DIM} \lceil log_2 \frac{4^{\mathbf{m}}}{n_{DIM}} \rceil + 3n_{DIM} \right)$$
$$\Rightarrow bpk_R = \frac{n_{DIM}}{n_{kmers}} \cdot (\lceil 2\mathbf{m} - log_2 n_{DIM} \rceil + 3) \tag{4.28}$$

argue in experiments section, that n_{kmers} has only a small effect on the size of R and can thus be disregarded as it is not needed for anything else

start experimental

Bit Vector Rank Query A bitvector of length U with n 1s can be encoded with $n\lceil log_2 U/n \rceil + 3n$ many bits supporting rank in $O(log_2(U/n)$ time [OS07].

$$T_{contains_R} = log_2 \frac{4^{\mathbf{m}}}{n_{DIM}} = 2\mathbf{m} - log_2 n_{DIM}$$

end experimental

Bitvector S has length $|O|+1 \approx |O|$ where every bit corresponds to the offset of one minimizer encountered in the sequences and one extra bit marks the end of all offsets - this last bit can be disregarded when modeling bpk_S as its effect is insignificantly small. Since $\mathbf{F}_K$ describes the percentage of k-mers with *infrequent* minimizers and every minimizer covers an average of $(k - \mathbf{m} + 2)/2$ consecutive k-mers, we can model the bits per k-mer in S through:

$$bpk_S = \mathbf{F}_K \cdot \frac{2}{k - \mathbf{m} + 2} \mathbf{OR} \tag{4.29}$$
$$= \frac{\mathbf{F}_M \cdot n_{minimizers}}{n_{kmers}} \tag{4.30}$$

make n_{kmers} part of model - either through sum of products or simply product of pKs times number of k-mers in the genome

figure out how to model the ratio between k-mers and minimizers for fractured input sequences OR use $n_{minimizers}$?

start experimental

Bit Vector Rank Query A bitvector of length U with n 1s can be encoded with $n\lceil log_2 U/n \rceil + 3n$ many bits supporting rank in $O(log_2(U/n)$ time [OS07].

$$T_{sel_S} = log_2 \frac{\mathbf{F}_M \cdot n_{minimizers}}{n_{DIM}} = \mathbf{f_{avg}}$$

end experimental

The offset vector O contains one entry of size $log_2|T|$ for the offset of each *infrequent* minimizer.

[1] Note that dividing by the average frequency safe as its value is at least 1 per definition Equation (3.19)

Since bpk_S already describes the number of offsets per k-mer, we can model the bits per k-mer in O with:

$$bpk_O = bpk_S \cdot log_2|T| \tag{4.31}$$

The hash set H contains an entry $(key : q)$ for each distinct k-mer q with an infrequent minimizer, which are $1-\mathbf{F}_K$ of all k-mers in T. Each entry has size $size(key) = 64$ bit. We introduce the load factor α to the model, in order to account for empty buckets in the hash table. The bits per k-mer required for the hash table can thus be modeled by:

$$bpk_H = (1 - \mathbf{F}_K) \cdot \frac{64}{\alpha} \tag{4.32}$$

überall hash table durch hash set ersetzten

reminder to include somewhere that we set $\alpha = 0.75$ in practice and explain why that is a reasonable value

in the used gtl::flat_hash_set there is one byte of metadata for each entry. because of this, it should be $bpk_H = (1 - \mathbf{F}_K) \cdot \frac{64+8}{\alpha}$ - should I add this here or in experiments?

This only models correctly for the unitigs. Figure out how to take into account that k-mers can appear many times in the Input but will only be added once to the hashtable

Locate benötigt eine Liste mit allen offsets: $bpk_H = (1 - \mathbf{F}_K) \cdot \frac{(64+(log_2|T| \cdot frequency))}{\alpha}$

The total bits per k-mer $bpk_{total} \in \mathbb{R}_0^+$ can be modeled by simply adding up the space requirements of each component:

$$bpk_{total} \in \mathbb{R}_0^+ \tag{4.33}$$
$$bpk_{total} = bpk_T + bpk_R + bpk_S + bpk_O + bpk_H \tag{4.34}$$

4.2.3 Optimization Goal

Having defined all constraints regarding the lookup time and space consumption of RSHash, we can now construct a MINLP in order to find optimal level parameters.
We introduce three more external parameters to the model: The optimization goal $OptGoal$ which specifies whether to minimize $T_{lookup_{RSHash}}$ or bpk_{total} and upper bounds $Bound_{time}, Bound_{bpk}$ for the lookup time and space consumption.

$$OptGoal \in \{T_{lookup_{RSHash}}, bpk_{total}\}$$
$$Bound_{time} \in \mathbb{R}_0^+$$
$$Bound_{bpk} \in \mathbb{R}_0^+$$

When using the MINLP, the upper bound belonging to the optimization goal should be set to ∞. In addition to the already defined ones, the following two constraints are added to the MINLP to enforce these bounds:

$$T_{lookup_{RSHash}} \leq Bound_{time} \tag{4.35}$$
$$bpk_{total} \leq Bound_{bpk} \tag{4.36}$$

One could for example search for level parameters minimizing the lookup time whilst not exceeding eight bits per k-mer by setting $OptGoal \leftarrow T_{lookup_{RSHash}}, Bound_{time} \leftarrow \infty, Bound_{bpk} \leftarrow 8$. The constructed MINLP for this example is:

$$
\begin{aligned}
INPUT : \; & OptGoal = T_{lookup_{RSHash}}, Bound_{time} = \infty, Bound_{bpk} = 8, \\
& \mathcal{L} = \{1\}, \mathcal{M}, \mathcal{T}, \\
& F_{M[(m,t)]}, F_{K[(m,t)]}, f_{avg}((m,t)) \; \forall (m,t) \in \mathcal{M} \times \mathcal{T}, \\
& T_{contains_R}, T_{sel_S}, T_{contains_O}, T_{comp_T}, T_{check_H}
\end{aligned}
$$

include $n_{minimizers}$ and n_{kmers}!!! and make them dependent of the level

MINIMIZE

$$OptGoal = T_{lookup_{RSHash}}$$

SUBJECT TO

$$\sum_{m \in \mathcal{M}} \mu_m = 1 \qquad \textit{Equation (4.3)}$$

$$\sum_{t \in \mathcal{T}} \tau_t = 1 \qquad \textit{Equation (4.4)}$$

$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m \cdot m \qquad \textit{Equation (4.7)}$$

$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t \cdot t \qquad \textit{Equation (4.8)}$$

$$n_{minimizers} = \mathbf{???}$$

$$n_{kmers} = \mathbf{???}$$

$$\mathbf{F}_M = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} F_{M[(m,t)]} \cdot \mu_m \tau_t \qquad \textit{Equation (4.10)}$$

$$\mathbf{F}_K = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} F_{K[(m,t)]} \cdot \mu_m \tau_t \qquad \textit{Equation (4.13)}$$

$$\mathbf{f_{avg}} = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} f_{avg}((m,t)) \cdot \mu_m \tau_t \qquad \textit{Equation (4.14)}$$

$$T_{level} = 2T_{sel_S} + (\mathbf{f_{avg}} + 1)/2 \cdot (T_{contains_O} + T_{comp_T}) \qquad \textit{Equation (4.18)}$$

$$T_{lookup_{RSHash}} = T_{contains_R} + \mathbf{F}_K \cdot T_{level} + (1 - \mathbf{F}_K) \cdot T_{check_H} \qquad \textit{Equation (4.17)}$$

$$T_{lookup_{RSHash}} \leq Bound_{time} = \infty \qquad \textit{Equation (4.35)}$$

$$bpk_T = 2 \qquad \textit{Equation (4.24)}$$

$$n_{DIM} = \frac{\mathbf{F}_K \cdot n_{kmers}}{\mathbf{f_{avg}}} \cdot \frac{2}{k - \mathbf{m} + 2} \mathbf{OR}$$

$$= \frac{\mathbf{F}_M \cdot n_{minimizers}}{\mathbf{f_{avg}}} \qquad \textit{Equation (4.27)}$$

$$bpk_R = \frac{n_{DIM}}{n_{kmers}} \cdot (\lceil 2\mathbf{m} - log_2 n_{DIM} \rceil + 3) \qquad \textit{Equation (4.28)}$$

$$bpk_S = \mathbf{F}_K \cdot \frac{2}{k - \mathbf{m} + 2} \mathbf{OR}$$

$$= \frac{\mathbf{F}_M \cdot n_{minimizers}}{n_{kmers}} \qquad \textit{Equation (4.30)}$$

$$bpk_O = bpk_S \cdot log_2 |T| \qquad \textit{Equation (4.31)}$$

$$bpk_H = (1 - \mathbf{F}_K) \cdot \frac{64}{\alpha} \qquad \textit{Equation (4.32)}$$

$$bpk_{total} = bpk_T + bpk_R + bpk_S + bpk_O + bpk_H \qquad \textit{Equation (4.34)}$$

$$bpk_{total} \leq Bound_{bpk} = 8 \qquad \textit{Equation (4.36)}$$

VARIABLES

$\mu_m \in \{0,1\} \mid \forall m \in M$	*Equation* (4.1)
$\tau_t \in \{0,1\} \mid \forall t \in T$	*Equation* (4.2)
$\mathbf{m} \in \mathcal{M}$	*Equation* (4.5)
$\mathbf{t} \in \mathcal{T}$	*Equation* (4.6)
$n_{minimizers} \in$???	
$n_{kmers} \in$???	
$\mathbf{F}_M \in [0,1]$	*Equation* (4.9)
$\mathbf{F}_K \in [0,1]$	*Equation* (4.11)
$\mathbf{f_{avg}} \in \mathbb{R}_0^+$	*Equation* (4.12)
$T_{level} \in \mathbb{R}_0^+$	*Equation* (4.15)
$T_{lookup_{RSHash}} \in \mathbb{R}_0^+$	*Equation* (4.16)
$bpk_T \in \mathbb{R}_0^+$	*Equation* (4.19)
$bpk_R \in \mathbb{R}_0^+$	*Equation* (4.20)
$bpk_S \in \mathbb{R}_0^+$	*Equation* (4.21)
$bpk_O \in \mathbb{R}_0^+$	*Equation* (4.22)
$bpk_H \in \mathbb{R}_0^+$	*Equation* (4.23)
$n_{DIM} \in \mathbb{R}_0^+$	*Equation* (4.25)
$bpk_{total} \in \mathbb{R}_0^+$	*Equation* (4.33)

4.3 Model for Several Levels

teils veraltet - one level finalisieren und dann dementsprechend für several levels anpassen

In the following, we adapt the constraints defined for using only one level in Section 4.2 to model the use of $L \in \mathbb{N}^+$ levels with $\mathcal{L} = \{l \in \mathbb{N}^+ \mid l \leq L\}$ being the set of levels used[2].

When modeling several levels, we add a superscript l to variables in order to indicate there belonging to a level $l \in \mathcal{L}$ e.g. $\mathbf{m}^1, \mathbf{m}^2$ and $\mathbf{m}^3$ denote the optimal minimizer lengths on level $1, 2$ and 3.

We still assume, that the times $T_{contains_{R^l}}, T_{sel_{S^l}}, T_{contains_{O^l}}, T_{comp_T}, T_{check_H}$ are (practically) independent of the parameters $\mathbf{m}^l, \mathbf{t}^l$ and the level l.

reason in chap: experiments

Unique sets of binary indicator variables μ_m^l and τ^l are needed for the level parameters $\mathbf{m}^l$ and $\mathbf{t}^l$ of each level l. The constraints regarding these variables are analog to those

[2] We model the space consumption and lookup time for any positive natural number of levels even though RSHash currently only supports the use of one, two or three levels [SMR26].

when using only one level:

$$
\begin{aligned}
\mu_m^l &\in \{0,1\} && \mid \forall (l,m) \in (\mathcal{L} \times \mathcal{M}) \\
\tau_t^l &\in \{0,1\} && \mid \forall (l,t) \in (\mathcal{L} \times \mathcal{T}) \\
\sum_{m \in \mathcal{M}} \mu_m^l &= 1 && \mid \forall l \in \mathcal{L} \\
\sum_{t \in \mathcal{T}} \tau_t^l &= 1 && \mid \forall l \in \mathcal{L} \\
\mathbf{m}^l &\in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
\mathbf{t}^l &\in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
\mathbf{m}^l &= \sum_{m \in \mathcal{M}} \mu_m^l \cdot m^l && \mid \forall l \in \mathcal{L} \\
\mathbf{t}^l &= \sum_{t \in \mathcal{T}} \tau_t^l \cdot t^l && \mid \forall l \in \mathcal{L}
\end{aligned}
$$

For each level l we model $\mathbf{F}_M^l$, $\mathbf{F}_K^l$ and $\mathbf{f}_{\mathbf{avg}}^l$ analogously to using a single level; however, it is necessary to sum over every possible combination of level parameters p^l. The number of additions grows exponentially for an increasing number of levels. We will outline the resulting consequences for solving the MINLP in Chapter 5.

don't forget to do this

$$
\begin{aligned}
\mathbf{F}_M^l &\in [0,1] && \mid \forall l \in \mathcal{L} \\
\mathbf{F}_K^l &\in [0,1] && \mid \forall l \in \mathcal{L} \\
\mathbf{f}_{\mathbf{avg}}^l &\in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
\mathbf{F}_M^l &= \sum_{p^l \in \mathcal{M}^l \times \mathcal{T}^l} F_{M[p^l]} \cdot \prod_{1 \le i \le l} \mu_{p^l[m^i]}^i \tau_{p^l[t^i]}^i && \mid \forall l \in \mathcal{L} \\
\mathbf{F}_K^l &= \sum_{p^l \in \mathcal{M}^l \times \mathcal{T}^l} F_{K[p^l]} \cdot \prod_{1 \le i \le l} \mu_{p^l[m^i]}^i \tau_{p^l[t^i]}^i && \mid \forall l \in \mathcal{L} \\
\mathbf{f}_{\mathbf{avg}}^l &= \sum_{p^l \in \mathcal{M}^l \times \mathcal{T}^l} f_{avg}(p^l) \cdot \prod_{1 \le i \le l} \mu_{p^l[m^i]}^i \tau_{p^l[t^i]}^i && \mid \forall l \in \mathcal{L}
\end{aligned}
$$

$p^l[t^l]$ in Notation einführen

4.3.1 Lookup Time

The search time on a level $l \in \mathcal{L}$ can be leveled analogously to that on one level:

$$
\begin{aligned}
T_{level}^l &\in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
T_{level}^l &= 2T_{sel_{S^2}} + \frac{\mathbf{f}_{\mathbf{avg}}^l + 1}{2} \cdot (T_{contains_{O^2}} + T_{comp_T}) && \mid \forall l \in \mathcal{L}
\end{aligned}
$$

For $\mathbf{F}_K^l$ of the k-mers on a level $l \in \mathcal{L}$ the lookup is completed by a search through the offsets on that level whilst the remaining $(1 - \mathbf{F}_K^l)$ of queries result in a lookup on the next level.

We can thus model the lookup time recursively:

$$T^l_{lookup_{RSHash}} = \begin{cases} T_{contains_{R^l}} + \mathbf{F}^l_K \cdot T^l_{level} \\ \qquad + (1 - \mathbf{F}^l_K) \cdot T^{l+1}_{lookup_{RSHash}} & \mid l \in \mathcal{L} \\ T_{check_H} & \mid l \notin \mathcal{L} \end{cases}$$

Considering the set $\mathcal{L} = \{1, 2\}$ of two levels as an example, this results in the following total lookup time:

$$\begin{aligned} T^1_{lookup_{RSHash}} = T_{contains_{R^1}} &+ \mathbf{F}^1_K \cdot T^1_{level} \\ &+ (1 - \mathbf{F}^1_K) \cdot \Big(T_{contains_{R^2}} + \mathbf{F}^2_K \cdot T^2_{level} \\ &\qquad + (1 - \mathbf{F}^2_K) \cdot T_{check_H} \Big) \end{aligned}$$

Inserting T^1_{level} and T^2_{level} yields:

$$\begin{aligned} T^1_{lookup_{RSHash}} = T_{contains_{R^1}} &+ \mathbf{F}^1_K \cdot (2T_{sel_{S^1}} + \frac{\mathbf{f}^1_{\mathbf{avg}} + 1}{2} \cdot (T_{contains_{O^1}} + T_{comp_T})) \\ &+ (1 - \mathbf{F}^1_K) \cdot \Bigg(T_{contains_{R^2}} + \mathbf{F}^2_K \cdot (2T_{sel_{S^2}} + \frac{\mathbf{f}^2_{\mathbf{avg}} + 1}{2} \cdot (T_{contains_{O^2}} + T_{comp_T})) \\ &\qquad + (1 - \mathbf{F}^2_K) \cdot T_{check_H} \Bigg) \end{aligned}$$

insertion weglassen?

4.3.2 Space Consumption

The text T is only being stored once, so the constant approximation $bpk_T \approx 2$ still holds.

Bitvector R^l n^l_{DIM} on each level l and are modeled analogously to using only one level:

$$\begin{aligned} n_{DIM} &\in \mathbb{R}^+_0 && \mid \forall l \in \mathcal{L} \\ bpk_{R^l} &\in \mathbb{R}^+_0 && \mid \forall l \in \mathcal{L} \\ n_{DIM} &= \frac{\mathbf{F}^l_K \cdot n^l_{kmers}}{\mathbf{f}^l_{\mathbf{avg}}} \cdot \frac{2}{k - \mathbf{m}^l + 2} \mathbf{OR} \\ &= \frac{\mathbf{F}^l_M \cdot n^l_{minimizers}}{\mathbf{f}^l_{\mathbf{avg}}} && \mid \forall l \in \mathcal{L} \\ bpk^l_R &= \frac{n^l_{DIM}}{n_{kmers}} \cdot \left(\lceil 2\mathbf{m}^l - log_2 n^l_{DIM} \rceil + 3 \right) && \mid \forall l \in \mathcal{L} \end{aligned}$$

Bitvector S^l on each level l has length $|O^l| + 1 \approx |O^l|$. Only k-mers that have frequent minimizers on all previous levels but an *infrequent* minimizer on level l lead to an offset in O^l and thus to a bit in S^l:

$$bpk_{S^l} \in \mathbb{R}_0^+ \qquad | \forall l \in \mathcal{L}$$

$$bpk_{S^l} = \left(\prod_{i=1}^{l-1} 1 - \mathbf{F}_K^i \right) \cdot \mathbf{F}_K^l \cdot \frac{2}{(k - \mathbf{m}^l + 2)} \qquad | \forall l \in \mathcal{L}$$

OR analogously to on level

$$bpk_S^l = \frac{\mathbf{F}_M^l * n_{minimizers}^l}{n_{kmers}} \qquad | \forall l \in \mathcal{L}$$

adapt to whatever stays on one model

For levels $l > 1$ the input consist of the super-k-mers with frequent minimizers on the previous level. Because those fragments are fairly short (sometimes close to k bases), every minimizer NO LONGER covers an average of $(k - \mathbf{m} + 2)/2$ consecutive k-mers - Find a way to account for that and describe this observation somewhere! When optimizing for the unitigs, this is the case for every level including ther first.

The offset vector O^l for every level l is modeled analogously to using only one level:

$$bpk_{O^l} \in \mathbb{R}_0^+ \qquad | \forall l \in \mathcal{L}$$

$$bpk_{O^l} = bpk_{S^l} \cdot log_2|T| \qquad | \forall l \in \mathcal{L}$$

The hash table H only contains an entry for k-mers q, which have frequent minimizers on all levels. The bits per k-mer for the hash table can thus be modeled as

$$bpk_H = \left(\prod_{l=1}^{L} 1 - \mathbf{F}_K^l \right) \cdot \frac{64}{\alpha}$$

The total bits per k-mer for L levels only differ to those on one level, as there are now a vector R, S and O on each level:

$$bpk_{total} = bpk_T + \sum_{l=1}^{L} (bpk_{R^l} + bpk_{S^l} + bpk_{O^l}) + bpk_H$$

4.4 Locate

outline differences to lookup

- lookup time: check every offset instead of only an average of half of the offsets
- hash table instead of hash set - contains one entry (key:k-mer, value:offset) for each appearance of every k-mer (no longer distinct) - for the unitigs, this shouldn't have an effect

5 Experiments

We evaluate how suited the MINLP proposed in Chapter 4 is for finding optimal level parameters by analyzing how accurate the modeled lookup time and space consumption are compared to the actual lookup time measured and space needed when building RSHash. We then examine how well the model can be optimized using the SCIP solver [Ach+08; Ach09; Bes+23]. Whether the resulting parameters are actually optimal in practice is highly dependent on how accurate the model is in the first place.

5.1 Datasets

We use the same datasets as Pibiri [Pib22]. Those are the whole genomes of the atlantic cod (*Gadus morhua*), the common kestrel (*Falco tinnunculus*) and the whole GRCh38 human genome (*Homo sapiens*). This choice allows a direct comparison with Schulte-Mattler and Reinert [SMR26] who use the same datasets for benchmarking RSHash.

absatz kopiert von [SMR26] - ist das okay?

5.2 Considered Parameters

For the following experiments we use k-mers of length $k = 31$ and the largest number of levels $L = 3$ that is currently supported by RSHash.
RSHash currently works with thresholds up to 255. Technically, using the proposed MINLP to find optimal level parameters for a specified optimization goal, one would have to consider every minimizer lengths $1 \leq m < k$ and all thresholds $1 \leq t \leq 255$. However, using the proposed model requires computing the distributions of minimizers for each possible combination of parameters. In practice using 3 levels would require computing the distributions for $30^3 \cdot 255^3 \approx 448 \cdot 10^9$ combinations of level parameters which is not viable.

überprüfen und warum dann nicht auch 255 berücksichtigen??

Using the maximum threshold $t = 255$ for each of the datasets above, the percentage of *infrequent* minimizers of length 15 is below *percentage*. When searching for level parameters which lead to relatively low query times whilst also keeping the data structure fairly compact, minimizers lengths $m < 15$ are not viable as this would lead to a huge hash table. On the other hand, the percentage of *infrequent* minimizers of length 25 is above *percentage*. Whilst this would lead to a small hash table, the compression would be small, as each minimizer covers only approximately $(31 - 25 + 2)/2 = 4$ k-mers. Because of this, considering minimizer lengths between fifteen and twenty five should be sufficient for analyzing the model. To further reduce the number of parameter

percentage einfügen!!

percentage einfügen!!

begründung überarbeiten; approximation gilt nur für erstes level und eventuell

combinations we only consider uneven minimizer lengths.
Recall that cumulative distribution functions $F_{M[p^l]}$ and $F_{K[p^l]}$ have logarithmic growth. It follows that using exponentially increasing thresholds should be sufficient for analyzing the MINLP. Using a threshold $t < 32$ leads to only very few *infrequent* minimizers for each of the datasets above, making it an implausible parameter choice. We thus consider the thresholds $32, 64$ and 128.

zeigen/besc

why not 255??

$$k = 31 \tag{5.1}$$
$$\mathcal{M} = \{m \in \mathbb{N} \mid 15 \leq m \leq 25 \wedge m\%2 = 1\} \tag{5.2}$$
$$\mathcal{T} = \{32, 64, 128\} \tag{5.3}$$

With these sets of considered parameters, using 3 levels would result in $6^3 \cdot 3^3 = 5,832$ combinations of level parameters. We reduce this even further by considering only non-decreasing parameter values on subsequent levels for both the minimizer lengths and thresholds. That means each combination of level parameters $p^3 = (m^1, m^2, m^3, t^1, t^2, t^3)$ fulfills $m^i \leq m^i + 1$ and $t^i \leq t^i + 1$ for all $i \in \{1, 2\}$ which can be done because.

reason

explain the constraint enforcing this in the MINLP and how the sum over possible minimizer combinations changes

For a non-decreasing sequence of length k', chosen from n values, the number of possibilities is $\binom{n+k'-1}{k'}$. That means that when choosing three minimizer lengths out of six possible values and three thresholds out of three possible values, the total number of possible parameter combinations is $\binom{6+3-1}{3} \cdot \binom{3+3-1}{3} = 560$.

5.3 Computing the Model Input

In order to evaluate the accuracy of the proposed MINLP, we first show how to compute the model input for each of the datasets which we refer to as the set of sequences S_{p^0}. This is done for $L = 3$ levels and non-decreasing level parameters as described in Section 5.2.
The MINLP requires the numbers of k-mers n_{kmers} and minimizers $n_{minimizers}$, the cumulative distributions of minimizer frequencies $F_{M[p^l]}$ and $F_{K[p^l]}$ and the average frequencies $f_{avg}(p^l)$ for every non-decreasing combination of level parameters $p^l \in \mathcal{M}^l \times \mathcal{T}^l$ on every level $1 \leq l \leq 3$ as input.

make dependent of parameters and include in model

These stats can be brute-forced for every dataset considered by iterating over each non-decreasing combination of the specified level parameters and actually building RSHash with those parameters. Whilst there are more efficient approaches to only compute the model input, we chose this approach as it not only computes an accurate model input but also allows to measure space consumption of the data structures components and the operation times on each of these components which we will later use to assess the accuracy of the values modeled by the MINLP.
Specifically we measure the bits per k-mer of the vectors R, S and O on each level as

well as the bits per k-mer to store the text and hash table which are only stored once and not per level. We measure the operation times $T_{contains_{R^l}}$, $T_{sel_{S^l}}$, $T_{contains_{O^l}}$, T_{comp_T} and T_{check_H} needed for the contains query on bitvector R, the select queries on bitvector S, a comparison with the text T at a random offset and checking whether a key exists in the hash set H. We also measure the average lookup time T_{lookup}. These times are average times of performing each operation 100000 subsequent times.

Algorithm 1 Computation of Minimizer Distribution

Input: sequences S_{p^0}, max Level L, considered minimizer lengths and thresholds $\mathcal{M}, \mathcal{T}$

```
function statsComputation(S_{p^0}, L, M, T)
    for p^L ∈ M^L × T^L do
        if p^L.isDecreasing() then
            continue
        end if
        RSHash ← buildWithStats(S_{p^0}, p^L)
        n_kmers, n_minimizers, F_{M[p^l]}, F_{K[p^l]}, f_avg(p^l) ← RSHash.getDistributionStats()
        spaceConsumptions_{p^L} ← RSHash.getSpaceConsumptions()
        operationTimes_{p^L} ← RSHash.measureTimes(noSamples = 100000)
    end for
end function
```

Explain results of minimizer distribution across several levels

- higher levels distributions are highly dependent on previous parameters (hypothesis: highly repetitive regions are more likely to have *infrequent* minimizers on every level regardless of the hash functions used)
- probability functions of minimizer distributions always follow a log normal / log logistic distribution?? Because there cumulative probability function has logarithmic growth, considering exponentially growing thresholds like $2, 4, 8, 16, 32, 64, 128$ is a viable choice (the change between 2 and 4 is proportional (equal?) to that between 64 and 128)
- outlook: it might be possible to only compute a selection of distributions and extrapolate the remaining ones

5.4 Model Accuracy

We asses the accuracy of the lookup time and space consumption modeled by the MINLP for each of the datasets considered. To do this we first compute model input, the actual space consumption and the measured operation times for every non-decreasing combination of level parameters on three levels as outlined in Section 5.3.
For each sequence and each combination of the level parameters considered, we then

use run a modified version of the MINLP which forces the exact combination of level parameters. This can be achieved by adding the following constraints:

define constraints

$$constraints... \tag{5.4}$$

5.4.1 Lookup Time

5.4.2 Space Consumption

5.5 Limitations

The MINLP approach for finding optimal level parameters has the advantage of finding the optimal level parameters for the specified optimization goal.

This approaches plausibility is heavily dependent on the time needed to compute the model input as outlined Section 5.3. Especially the runtime for the computation of the distributions $p_{M[p^{l-1},p^l[m^l]]}$ and $p_{K[p^{l-1},p^l[m^l]]}$ for every combination of the level parameters p^l on every level l grows exponentially with the number of levels when done in a trivial way.

AHHHHH - Achtung das böse Wort!!!

We further outline this in Chapter 5.

For this reason, an approach yielding only an approximation of the optimal level parameters but taking less time to do so could be of interest.

We will further explore methods Chapter 7 the genome in order to reduce the computation of minimizer distribution.

TODO

In another approach we will try to approximate the distributions on higher levels based on that on the first level in order to get around the exponential cost of computing those values.

TODO

6 Outlook

7 Subsampling

7.1 Motivation

The time for processing a genome in order to find the minimizer distribution $\mathbf{F}_K(p^l)$ for possible minimizer lengths $m \in \mathcal{M}$ and thresholds $t \in \mathcal{T}$ is directly proportional to the size of the genome.
For large genomes, this processing can become the dominating factor when searching for optimal parameters using the Chapter 4 approach. In order to speed up the search for the empirical values, we introduce a subsampling method.

7.2 Subsampling Method

Instead of calculating the distribution over the full genome, we sample n sequences of length $l \geq k$ at random positions in the genome.

how to handle overlapping samples

müsste die gezählte frequenz der minimizer mit eine Faktor mutipliziert werden, um auf die länge des ursprünglichen textes zu skalieren?? schließlich kommen minimizer ja in einem kürzeren text perse seltener vor als in einem längeren

7.3 Time Complexity of Finding the Minimizer Distributions

genaue zeitliche abhängigkeit 1. verstehen und 2. aufschreiben

7.4 Time Complexity using Subsampling

By applying the proposed subsampling strategy, the original genome T of length $|T|$ is reduced to the subsampled text T^S of length $|T^S| = n \cdot l$. This means a reduction in size by a factor of $\frac{|T|}{|T^S|}$.
Depending on the choice of n and l this results in a speedup of the processing time by a factor of or even a constant processing time when both parameters are chosen as a constand regardless of $|T|$ (However, this should result in a growing error for larger genomes).

assuming no overlaps

factor

time

7.5 Error Analysis

7.6 Choosing l

The length l has to be at least k.

wenn $l > k$ ist, überlappen sich mehrere kmere an den gesampleten stellen.
Frage: ich verstehe noch nicht, warum genau dass von vorteil ist.
Antwort: Weil roling hash - l ca 1000, dann quasi linear

7.7 Choosing n

Choosing a constant n would result in a processing time that would no longer be dependent of the genomes length $|T|$, as the sampled text would have constant size $n \cdot l$. For large genomes, the percentage $\frac{|T^S|}{|T|}$ of the genome represented in the subsample can become arbitrarily small, resulting in growing error.
If instead n is chosen relative to $|T|$, $\frac{|T^S|}{|T|}$ will have a constant value independent of the genomes size. Even though this means, that the time needed, in order to find the minimizer distributions would still be proportional to the genomes size, this should be the preferred option as it results in a bounded error of the approximation.
For a large enough n, the probability of choosing a bad representation becomes highly unlikely.

assuming no overlaps

zu vage, begründen

8 stuff

A fairly straightforward way of computing these is to use a recursion over the levels. The input for each recursive call consist of the current level l, the number of levels L, the previous level parameters p^{l-1} and the set of input sequences in dependence of those parameters.

zu umgangssprachlich?

Upon being called, the first minimizer $m \in \mathcal{M}$ is selected and used to compute the distribution stats for the input sequence on this level in dependence of m. The function then loops over each threshold $t \in \mathcal{T}$, removes all k-mers with *infrequent* minimizers when using (m, t) from the input sequences and then calls the recursion for the next level, which receives the next level, the previous parameters extended by (m, t) and the modified sequences as input. Only once this recursion is completed for every threshold $t \in \mathcal{T}$, the next minimizer is picked from $\mathcal{M}$. The described algorithm:

soll ich hier erklären, wie die Berechnung in der Praxis noch ein bisschen schneller geschieht? Oder ist das an dieser Stelle egal, wenn es am grundlegenden Problem der exponentiellen Laufzeit nichts ändert?

Algorithm 2 Computation of Minimizer Distribution

Input: level l, max Level L, level parameters p^{l-1}, sequences $S_{p^{l-1}}$

```
function RecursiveMinimizerDistribution(l, L, p^{l-1} S_{p^{l-1}})
    if l = L then
        return
    end if
    for m ∈ M do
        for t ∈ T do
            p^l ← extend(p^{l-1}, m, t)            ▷ extend((m^1, t^1), m^2, t^2) = (m^1, m^2, t^1, t^2)
            S_{p^l} ← getFrequentKmers(l, S_{p^{l-1}}, m, t)

            n_kmers, n_minimizers ← getCounts(h_l, S_{p^{l-1}}, m)
            F_{M[p^l]}, F_{K[p^l]}, f_avg(p^l) ← distributionStats(h_l, S_{p^{l-1}}, m)
            RecursiveMinimizerDistribution(l + 1, L, p^l, S_{p^l})
        end for
    end for
end function
```

add $n_{minimizers}, n_{kmers}$ to model and include dependence of level parameters

This approach is guaranteed to compute correct minimizer distributions for every set of level parameters on every level as it effectively performs the same steps as the

RSHash algorithm only without building the data structure itself. This also means, that one could include the building of bitvectors R, S and O on every level as well as the hash set H and simply measure the space consumption and lookup time for every combination, which would render the MINLP obsolete.
However, this approach is sufficient for computing the input needed for analyzing how well the MINLP models the lookup time and space consumption using only the minimizer distributions. Possibilities of speeding up the time needed are outlined in Chapter 6.

Bibliography

[Ach+08] T. Achterberg, T. Berthold, T. Koch, and K. Wolter. *Constraint Integer Programming: a New Approach to Integrate CP and MIP*. eng. Tech. rep. 08-01. Takustr. 7, 14195 Berlin: ZIB, 2008 (cit. on p. 28).

[Ach09] T. Achterberg. "SCIP: solving constraint integer programs". In: *Mathematical Programming Computation* 1.1 (2009), pp. 1–41. URL: https://doi.org/10.1007/s12532-008-0001-1 (cit. on p. 28).

[Bes+23] K. Bestuzheva, A. Chmiela, B. MÃ¼ller, F. Serrano, S. Vigerske, and F. Wegscheider. *Global Optimization of Mixed-Integer Nonlinear Programs with SCIP 8*. Tech. rep. 2301.00587. arXiv, 2023 (cit. on p. 28).

[Chi+14] R. Chikhi, A. Limasset, S. Jackman, J. Simpson, and P. Medvedev. *On the representation of de Bruijn graphs*. en. arXiv:1401.5383 [q-bio.QM]. Oct. 2014. URL: http://arxiv.org/abs/1401.5383 (visited on 09/18/2026) (cit. on p. 7).

[Jai+20] C. Jain, A. Rhie, H. Zhang, C. Chu, B. P. Walenz, S. Koren, and A. M. Phillippy. "Weighted minimizer sampling improves long read mapping". en. In: *Bioinformatics* 36.Supplement_1 (July 2020), pp. i111–i118. URL: https://academic.oup.com/bioinformatics/article/36/Supplement_1/i111/5870473 (visited on 09/18/2026) (cit. on p. 7).

[Moe+24] C. Moeckel, M. Mareboina, M. A. Konnaris, C. S. Chan, I. Mouratidis, A. Montgomery, N. Chantzi, G. A. Pavlopoulos, and I. Georgakopoulos-Soares. "A survey of k-mer methods and applications in bioinformatics". en. In: *Computational and Structural Biotechnology Journal* 23 (Dec. 2024), pp. 2289–2303. URL: https://spj.science.org/doi/10.1016/j.csbj.2024.05.025 (visited on 11/02/2026) (cit. on p. 6).

[OS07] D. Okanohara and K. Sadakane. "Practical entropy-compressed rank/select dictionary". In: *2007 Proceedings of the ninth workshop on algorithm engineering and experiments (ALENEX)*. SIAM. 2007, pp. 60–70 (cit. on pp. 15, 19, 20).

[Pib22] G. E. Pibiri. "Sparse and skew hashing of K-mers". en. In: *Bioinformatics* 38.Supplement_1 (June 2022), pp. i185–i194. URL: https://academic.oup.com/bioinformatics/article/38/Supplement_1/i185/6617506 (visited on 07/06/2026) (cit. on pp. 7, 9, 28).

[SMR26] J. Schulte-Mattler and K. Reinert. "RSHash: a fast and space-efficient hash table for k -mers". en. In: *Bioinformatics* 42.Supplement_2 (Aug. 2026), btag440. URL: https://academic.oup.com/bioinformatics/article/doi/10.1093/bioinformatics/btag440/8767308 (visited on 09/15/2026) (cit. on pp. 7, 8, 12, 18, 24, 28).

[ZKM20] H. Zheng, C. Kingsford, and G. Marçais. "Improved design and analysis of practical minimizers". en. In: *Bioinformatics* 36.Supplement_1 (July 2020), pp. i119–i127. URL: https://academic.oup.com/bioinformatics/article/36/Supplement_1/i119/5870484 (visited on 07/06/2026) (cit. on pp. 7, 9).